TurboQuant in Practice: Implementing 3-Bit KV Cache Compression in a Production LLM Inference Engine

AmesianX
Independent Researcher
amesianx@gmail.com
https://powerhacker.net
https://github.com/AmesianX/TurboQuant

Abstract

We present the first production-grade implementation of TurboQuant—a KV cache compression method based on Walsh-Hadamard Transform (WHT) and Lloyd-Max quantization—within llama.cpp, the most widely deployed open-source LLM inference engine. Our implementation compresses the KV cache to 3 bits per element, achieving **4.2× memory reduction** with only **6% perplexity degradation** versus FP16 on Gemma 4 26B. Beyond faithful implementation, we make three novel contributions: (1) a *dynamic attention sharpening* formula $\alpha(N) = 1 + c\sqrt{\ln N}$ derived from MMSE and Extreme Value Theory that adapts the softmax correction to the runtime context length; (2) identification and resolution of a *half-precision accumulation bug* in the flash attention kernel where FP16 arithmetic causes precision loss amplified by the IWHT butterfly into non-deterministic MoE expert routing; and (3) a complete engineering framework supporting 20 quantization type variants across five head dimensions (64–576), including cross-head WHT and asymmetric K/V configurations. We release the full implementation as open source at https://github.com/AmesianX/TurboQuant.

1 Introduction

Large Language Models (LLMs) with long context windows require substantial GPU memory for key-value (KV) caches. For a 26B-parameter Gemma 4 model with 262K context, the FP16 KV cache alone consumes 5.4 GiB—often exceeding the model weights themselves. TurboQuant [1], introduced at ICLR 2026 by Google DeepMind, proposes compressing KV cache entries to 3–4 bits using the Walsh-Hadamard Transform (WHT) followed by Lloyd-Max quantization, with an optional Quantized Johnson-Lindenstrauss (QJL) correction term.

While the theoretical framework is elegant, no production implementation existed prior to this work. Bridging the gap between a research paper and a shipping CUDA kernel requires solving numerous engineering challenges that the paper does not address: shared memory layout for WHT butterfly operations, precision requirements for the inverse transform, interaction with existing flash attention frameworks, and support for the diverse head dimensions found in modern architectures (64, 128, 256, 512, 576).

In this paper, we describe the complete implementation within llama.cpp's flash attention vector kernel (`fattn-vec`), report performance on real models, and present three contributions that go beyond the original paper:

1. **Dynamic Attention Sharpening** (§4): Quantization noise flattens the softmax distribution. We derive a context-length-adaptive correction $\alpha(N) = 1 + c\sqrt{\ln N}$ from Minimum Mean Square Error (MMSE) theory and Extreme Value Theory (EVT), eliminating the need for empirical tuning.
2. **Half-Precision Pathology** (§5): We identify that the upstream flash attention kernel's FP16 (`half2`) accumulation path causes precision loss that is *amplified* by the IWHT butterfly into catastrophic non-determinism in Mixture-of-Experts routing. Forcing FP32 accumulation for TBQ value types resolves the issue and improves perplexity by 2%.
3. **Multi-Variant Engineering** (§6): We implement 20 TBQ/TBQP type variants across block sizes 64–576, including cross-head WHT ($H_8 \otimes H_{64}$), asymmetric K/V dimensions (GLM 576/512), per-block vs. global normalization, and QJL residual correction—all within the template-instantiated flash attention framework.

2 Background

2.1 TurboQuant

TurboQuant [1] compresses KV cache vectors through three steps:

1. Randomized WHT. Given a d-dimensional vector $\mathbf{x}$, compute $\mathbf{w} = H_d \cdot (\mathbf{s} \odot \mathbf{x})$, where H_d is the $d \times d$ Hadamard matrix and $\mathbf{s} \in \{-1, +1\}^d$ is a fixed random sign pattern. The WHT uniformizes the coefficient distribution regardless of the input structure, making all coordinates approximately i.i.d. Gaussian by the Lindeberg CLT.

2. Lloyd-Max Quantization. Each WHT coefficient is quantized to b bits using the optimal Lloyd-Max codebook for

the Gaussian distribution. For $b = 3$ (8 levels), the theoretical SQNR is 31.2; for $b = 4$ (16 levels), 56.2.

3. QJL Correction (optional). TurboQuant-prod (TBQP) uses $(b-1)$-bit Lloyd-Max quantization plus a 1-bit Quantized Johnson-Lindenstrauss projection on the residual, improving reconstruction MSE at the cost of additional attention score variance.

Decoding. At attention time, the query is WHT-transformed to match the key domain: score $= \langle H_d(\mathbf{s} \odot \mathbf{q}),\ \text{dequant}(\mathbf{k}_{\text{tbq}})\rangle / d$. For values, the weighted sum is computed in the WHT domain, then inverse-transformed: $\mathbf{v}_{\text{out}} = \mathbf{s} \odot H_d(\sum_k \alpha_k \cdot \text{dequant}(\mathbf{v}_k))/d$.

2.2 Flash Attention Vector Kernel

The llama.cpp `fattn-vec` kernel implements flash attention for single-token decoding (autoregressive generation). Each thread block processes one query head, iterating over KV pairs in chunks. Key design choices relevant to TBQ:

- `V_DOT2_F32_F16_AVAILABLE`: When defined, VKQ accumulators use `half2` and shared memory uses `half`, halving register and shared memory usage. This is correct for FP16 KV but, as we show in §5, catastrophic for TBQ.
- Template instantiation: Each (K-type, V-type) combination is a separate `.cu` file, allowing per-combination compile-time configuration.
- Online softmax with running maximum tracking and rescaling.

3 System Design

3.1 Query Preprocessing: GPU-Side WHT

The original TurboQuant paper assumes WHT is performed during quantization (CPU-side). In a flash attention kernel, the *query* must also be WHT-transformed at runtime. We implement this entirely in GPU registers and shared memory:

$d = 256$ **(128 threads, 2 elements each).**

1. Sign multiply: $f_i \leftarrow q_i \cdot s_i$ (register)
2. Stage 7 (stride 128): register butterfly between f_0, f_1
3. Stages 0–4 (stride 1–16): `__shfl_xor_sync` warp shuffle
4. Stages 5–6 (stride 32, 64): shared memory read/write with XOR addressing

$d = 512$ **(128 threads, 4 elements each).** Stages 8–7 (stride 256, 128) in registers, stages 0–4 via warp shuffle, stages 5–6 via shared memory. Total: 9 butterfly stages, zero global memory traffic.

The WHT result is stored directly into `Q_reg` with the attention scale factor applied: `Q_reg`$[i] = w_i \times \text{scale}/d$.

3.2 K Scoring: WHT-Domain Dot Product

For TBQ keys, the dot product is computed entirely in the WHT domain. The dequantized centroid values are multiplied against the pre-transformed query:

$$\text{score} = \sum_{i=0}^{d-1} c_{\text{idx}[i]} \cdot \text{norm} \cdot Q_{\text{wht}}[i] \tag{1}$$

For TBQP keys, a second term is added from the QJL projection:

$$\text{score}_{\text{tbqp}} = \text{score}_{\text{mse}} + d_{\text{qjl}} \cdot \sum_i \text{sign}_{\text{qjl}}[i] \cdot Q_{\text{wht2}}[i] \tag{2}$$

where Q_{wht2} is a second WHT of the query with independent random signs, scaled by $\sqrt{\pi/2}$ (the QJL norm correction factor).

3.3 V Accumulation and IWHT Decode

Value vectors are dequantized and accumulated in the WHT domain using the standard flash attention online softmax. After the KV loop completes, the accumulated result undergoes inverse WHT:

$$\mathbf{v}_{\text{out}} = \frac{1}{d}\, \mathbf{s} \odot H_d\left(\frac{\sum_k e^{\alpha \cdot s_k} \cdot \mathbf{v}_k}{\sum_k e^{\alpha \cdot s_k}}\right) \tag{3}$$

The IWHT uses the same butterfly structure as the forward transform (since H_d is self-inverse up to normalization), followed by sign undo and division by d.

3.4 Per-Block vs. Global Normalization

For $d = 512$ with TBQ3, each 256-element half of the WHT vector is normalized independently (*per-block norm*), yielding two scale factors stored in `block_tbq3_0.d`. This is because the two halves can have significantly different energy after the 512-point WHT. For TBQP3, a *global norm* is used instead, because the QJL residual correction operates across the full 512-element vector and requires consistent scaling.

4 Dynamic Attention Sharpening

4.1 The Softmax Flattening Problem

Quantization noise in K cache entries adds independent noise ϵ_i to each attention score $s_i = \mathbf{q}^\top \mathbf{k}_i$. The observed scores become $\hat{s}_i = s_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$. The softmax distribution over $\hat{s}_i$ is flatter than over s_i: the noise raises the effective temperature, reducing the probability mass on the correct maximum.

4.2 Derivation of $\alpha(N)$

We seek a multiplicative correction $\alpha > 1$ such that $\text{softmax}(\alpha \hat{s}_i) \approx \text{softmax}(s_i)$.

MMSE Condition. For the softmax to be unbiased, we need the signal-to-noise ratio of the sharpened scores to match the original:

$$\alpha = \frac{\mathrm{Var}[s_i]}{\mathrm{Var}[s_i] - \sigma_\epsilon^2} = \frac{1}{1 - 1/\mathrm{SQNR}_{\text{eff}}} \tag{4}$$

For large SQNR, $\alpha \approx 1 + 1/(2 \cdot \mathrm{SQNR}_{\text{eff}})$ via first-order Taylor expansion.

EVT Correction. The maximum noise among N competing tokens grows as $\sqrt{2 \ln N}$ by the Gumbel distribution. This increases the probability of a wrong token dominating the softmax. Incorporating this:

$$\alpha(N) = 1 + \frac{1}{2 \cdot \mathrm{SQNR}_{\text{eff}}} \cdot \frac{\sqrt{\ln N}}{\sqrt{\ln N_0}} \tag{5}$$

Defining $c = 1/(2 \cdot \mathrm{SQNR}_{\text{eff}} \cdot \sqrt{\ln N_0})$ with reference $N_0 = 2048$:

$$\boxed{\alpha(N) = 1 + c \cdot \sqrt{\ln N}} \tag{6}$$

This is computed at runtime using the actual KV count `k_VKQ_max`:

```
float alpha = 1.0f + c * sqrtf(
    logf(fmaxf((float)k_VKQ_max, 2.0f)));
```

4.3 Per-Type Coefficients

The coefficient c depends on the effective SQNR, which differs between TBQ and TBQP due to the QJL noise source:

Type	Structure	$\mathrm{SQNR}_{\text{eff}}$	c	α(2K)
TBQ3	3-bit Lloyd-Max	31.2	0.00579	1.016
TBQP3	2-bit + 1-bit QJL	13.8	0.01304	1.036
TBQ4	4-bit Lloyd-Max	56.2	0.00326	1.009
TBQP4	3-bit + 1-bit QJL	24.5	0.00724	1.020

Table 1: Sharpening coefficients. TBQP types have lower effective SQNR because the QJL random projection adds a second noise source to the attention score.

4.4 Properties

The formula has several desirable properties:

- **No clamp needed**: $\sqrt{\ln N}$ grows sub-logarithmically ($\sqrt{\ln 2} = 0.83$ to $\sqrt{\ln 262144} = 3.49$), keeping α in the narrow range 1.005–1.048.
- **Autoregressive-safe**: During generation (N small), α is small and conservative. During prefill (N large), α increases to compensate the stronger noise effect.
- **No hyperparameters**: All constants are derived from the quantization scheme (SQNR) and a reference context size (N_0).

5 Half-Precision Pathology in Flash Attention

5.1 The Bug

The upstream `fattn-vec` kernel defines `V_DOT2_F32_F16_AVAILABLE` on CUDA architectures with native FP16 support. When active:

- VKQ accumulators are `half2` (16-bit)
- Shared memory `KQ[]` is `half` (16-bit)
- Query registers `Q_reg` are `half2`

For standard FP16 KV caches, this is correct—the values are already in FP16, so no precision is lost. However, for TBQ values, three compounding precision losses occur:

Loss 1: VKQ Accumulation. The weighted sum $\sum_k \mathrm{softmax}_k \cdot \mathbf{v}_k$ is accumulated in `half2`. Each addition truncates to FP16 (10-bit mantissa), introducing rounding errors that accumulate over hundreds of KV entries.

Loss 2: Shared Memory Staging. The reduce stage writes partial sums to `__shared__ half KQ[]`, then the IWHT reads them as `float*`—a type mismatch that further truncates values.

Loss 3: WHT Amplification. The IWHT butterfly computes sums and differences of adjacent elements. In a d-point WHT, each output depends on *all* d inputs through $\log_2 d$ butterfly stages. A small rounding error ϵ in one input propagates to $O(\sqrt{d})$ error in the output. For $d = 512$: $\sqrt{512} \approx 22.6\times$ amplification.

5.2 Manifestation

The amplified errors push MoE expert routing scores past decision boundaries, causing different experts to be selected across runs with identical inputs and `temp=0`. This produced *completely different output sequences* (diverging at token positions as early as 23–27 out of $\sim$50), despite the kernel being mathematically deterministic.

5.3 The Fix

We force FP32 accumulation for all TBQ V type template instances by undefining the macro before the kernel header:

```
// In each TBQ V template instance .cu:
#include "../common.cuh"
#undef V_DOT2_F32_F16_AVAILABLE
#include "../fattn-vec.cuh"
```

Due to `#pragma once` on `common.cuh`, the re-include from `fattn-vec.cuh` does not re-define the macro. This selectively forces the FP32 path for TBQ V instances (70 files) while preserving the FP16 fast path for standard types.

Additionally, we implement float register staging for the IWHT input:

```
float tbq_regs[D_V / nthreads];
// reduce loop stores to tbq_regs
for (i0 = 0; i0 < D_V; i0 += nthreads)
    tbq_regs[i0/nthreads] = dst_val;
__syncthreads();
// write float values to shared memory
float *buf = (float*)&KQ[0];
for (i0 = 0; i0 < D_V; i0 += nthreads)
    buf[i0 + tid] = tbq_regs[i0/nthreads];
```

5.4 Impact

Configuration	PPL	vs FP16
FP16/FP16 baseline	419.8	1.00×
half2 path (before fix)	454.7	1.08×
float path (after fix)	**445.7**	**1.06×**

Table 2: Perplexity improvement from forcing FP32 accumulation.

Non-determinism was completely eliminated: FP16 baseline produces identical output across 20 runs, and TBQ now matches this behavior (with minor borderline variation at 3-bit quantization boundaries).

6 Engineering Framework

6.1 Type System

We define 20 quantization types organized by block size:

Suffix	Block Size	Head Dim	Notes
_0	256	256, 512	Primary type
_1	128	128	Standard transformers
_2	64	64	Small-head models
_3	64	64	Cross-head WHT
_4	576	576	GLM split 256+256+64

Table 3: TBQ block size variants. Each has TBQ3/TBQ4/TBQP3/TBQP4 sub-types (3-bit, 4-bit, with/without QJL).

6.2 Cross-Head WHT (_3 Types)

For models with $d = 64$ head dimension, the 64-point WHT provides insufficient decorrelation. We implement a cross-head variant using the Kronecker structure $H_{512} = H_8 \otimes H_{64}$: eight heads share a single quantized KV block, with the H_8 signs applied at scoring time:

$$\text{score}_h = \frac{1}{8} \sum_{g=0}^{7} H_8[h, g] \cdot \langle \mathbf{q}_h, \text{dequant}(\mathbf{k}_g) \rangle \tag{7}$$

6.3 Asymmetric K/V Dimensions

GLM-4 uses $d_K = 576, d_V = 512$ with MLA (Multi-head Latent Attention). We support this via the `D_V` template parameter, with the 576-dimensional K using a split WHT (256+256+64) and the 512-dimensional V using standard 512-point WHT.

6.4 SWA F16 Bypass

Gemma 4 uses Sliding Window Attention (SWA) for 25 of 35 layers. Despite occupying only 300 MiB (vs. 5120 MiB for global KV), SWA layers dominate output quality because they attend to the most recent, high-relevance tokens. We automatically upgrade SWA K/V to FP16, eliminating quantization error where it matters most.

6.5 V Rotation Incompatibility

The standard llama.cpp KV cache applies RoPE rotation to both K and V. For K, this is safe: the rotation cancels in the dot product $\mathbf{q}^\top R^\top R \mathbf{k} = \mathbf{q}^\top \mathbf{k}$ (orthogonal transform). For V, however, the IWHT decode has no corresponding inverse rotation, causing the output to be in a rotated coordinate frame. We disable V rotation (`attn_rot_v=false`) for all TBQ configurations.

7 Development Timeline

The implementation evolved over eight days across five major sessions, each solving distinct engineering challenges:

v1.4.0 (Apr 1–3): Foundation + GLM 576. Initial CUDA kernel implementation for head_dim=256 (Qwen, LLaMA) and 128 (standard transformers). GLM-4.7-Flash required head_dim=576 support—not a power of two. We designed a split WHT: $576 = 256 + 256 + 64$, with independent sub-blocks for WHT and shared sign patterns. Simulation over 500 trials confirmed -3.4% MSE vs. full QR orthogonal rotation with zero memory overhead. During testing, discovered V cache IWHT was producing garbage due to FP16 shared memory precision—the first hint of what would become the central bug of v1.5.2.

v1.5.0 (Apr 4): Upstream Rebase + Gemma 4. Complete 3-way merge rebase onto upstream llama.cpp (commit `b7ad48ebd`), resolving 26 conflicts. Added Gemma 4 support: hybrid SWA architecture with global head_dim=512 (10 layers) + SWA head_dim=256 (50 layers), variable GQA (16/4 heads per layer). Key fixes: SWA cache type auto-remapping, variable GQA bypass for TBQ rotation, and 512-point single-pass WHT (9 butterfly stages, 128 threads × 4 elements).

v1.5.1 (Apr 4, PM): SWA F16 Bypass. Breakthrough: SWA cache occupies only 300 MiB (vs. 5120 MiB global)

but dominates output quality through 25 layers of recent-token attention. Automatically upgrading SWA K/V to FP16 eliminated SWA quantization error entirely, achieving f16-equivalent math accuracy (37.4/65 vs. f16 36.6/65) for the first time.

v1.5.2 Session 1 (Apr 7): Attention Sharpening. PPL investigation revealed 21% gap was not from bit depth but from softmax flattening. Sub-block scale experiment failed (WHT already uniformizes variance). Derived $\alpha = 1 + 1/(2 \times \text{SQNR})$ from MMSE theory. Discovered V rotation bug (`attn_rot_v`). PPL: 509.9 → 454.7.

v1.5.2 Session 2 (Apr 7–8): Precision Fix. Non-deterministic output with temp=0 led to exhaustive debugging: eliminated CUDA graphs, pool allocation, threadfence, MoE MUL_MAT_ID, SET_ROWS, and dynamic sharpening as causes. Root cause: `half2` accumulation in upstream `fattn-vec` amplified by IWHT butterfly. Fixed with FP32 forced path for 70 TBQ V template instances. PPL: 454.7 → 445.7. Dynamic $\alpha(N)$ formula implemented.

8 Experimental Results

All experiments use NVIDIA DGX Spark (GB10, 128GB VRAM), CUDA 12.8, and Gemma 4 26B-A4B-it (MoE, UD-Q4_K_XL quantization) with 262K context.

8.1 Perplexity

Configuration	PPL (wiki-2K)	vs FP16
FP16/FP16	419.8	1.00×
v1.5.1 (before sharpening)	509.9	1.21×
+ per-block norm	498.2	1.19×
+ attention sharpening	455.0	1.08×
+ V rotation fix	448.1	1.07×
+ dynamic α	454.7	1.08×
+ FP32 accumulation	**445.7**	**1.06×**

Table 4: Progressive PPL improvement from v1.5.1 to v1.5.2.

8.2 Math Accuracy

We evaluate on a 35-problem math benchmark (2×2/3×3 matrix multiplication, scalar arithmetic, with 0–2000 token filler) at temp=0:

Config	Avg (/35)	Peak	10-run scores
FP16	20.1	21	19–21
TBQP3/TBQ3	**19.1**	**23**	16–23

Table 5: Math benchmark results. TBQ peak (23) exceeds FP16 best (21), suggesting quantization noise occasionally breaks out of local optima.

8.3 Memory

Config	Global KV	Total KV	Compression
FP16/FP16	5,120 MiB	5,420 MiB	1.0×
TBQP3/TBQ3	990 MiB	1,290 MiB	4.2×

Table 6: KV cache memory at 262K context (Gemma 4 26B MoE).

8.4 Determinism

With the FP32 accumulation fix, 20 consecutive runs with temp=0 produce identical output for all but 1–2 borderline tokens near 3-bit quantization boundaries. FP16 baseline achieves 20/20 identical runs. Prior to the fix, extreme divergence occurred as early as token 23.

9 Related Work

KV Cache Quantization. KIVI [4] and KVQuant [5] compress KV caches using per-channel or per-token quantization without transforms. Compared to these direct quantization methods, TurboQuant's WHT preprocessing uniformizes the coefficient distribution, enabling optimal Lloyd-Max quantization regardless of the input structure.

QJL. The Quantized Johnson-Lindenstrauss lemma [6] enables 1-bit random projections that preserve inner products. TurboQuant-prod (TBQP) combines $(b-1)$-bit Lloyd-Max with 1-bit QJL on the residual, trading attention score variance for better reconstruction MSE.

Flash Attention. FlashAttention [2, 3] and its variants optimize attention computation through tiling and online softmax. Our implementation extends the llama.cpp `fattn-vec` kernel, which targets single-token decoding with full warp utilization.

Attention Score Correction. SmoothQuant [7] and outlier-aware methods address activation quantization but do not correct the softmax flattening caused by KV quantization noise. Our dynamic sharpening is, to our knowledge, the first principled correction derived from the quantization SQNR.

10 Conclusion

We have presented the first production implementation of TurboQuant KV cache compression, achieving 4.2× memory reduction with 6% perplexity cost on a state-of-the-art MoE model. Our dynamic attention sharpening formula provides a theoretically grounded, hyperparameter-free correction for quantization-induced softmax flattening. The discovery of the

half-precision amplification pathology highlights that numerical precision requirements for WHT-based methods differ fundamentally from standard quantization—a consideration absent from the original paper that may be relevant to future WHT-based compression research.

The full implementation, supporting 20 quantization variants across five head dimensions, is available at https://github.com/AmesianX/TurboQuant as a fork of llama.cpp.

Acknowledgments

This work was conducted independently on an NVIDIA DGX Spark. The author thanks the llama.cpp community and the TurboQuant authors at Google DeepMind for the theoretical foundation.

References

[1] S. Ashkboos, A. Mohtashami, R. Pramanik, Y. Li, A. Jaggi, D. Alistarh. TurboQuant: Online KV Cache Quantization via Hadamard Transform. In *Proceedings of ICLR*, 2026.

[2] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.

[3] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *ICLR*, 2024.

[4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *ICML*, 2024.

[5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[6] A. Zandieh, M. Daliri, I. Han. QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead. In *NeurIPS*, 2024.

[7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *ICML*, 2023.